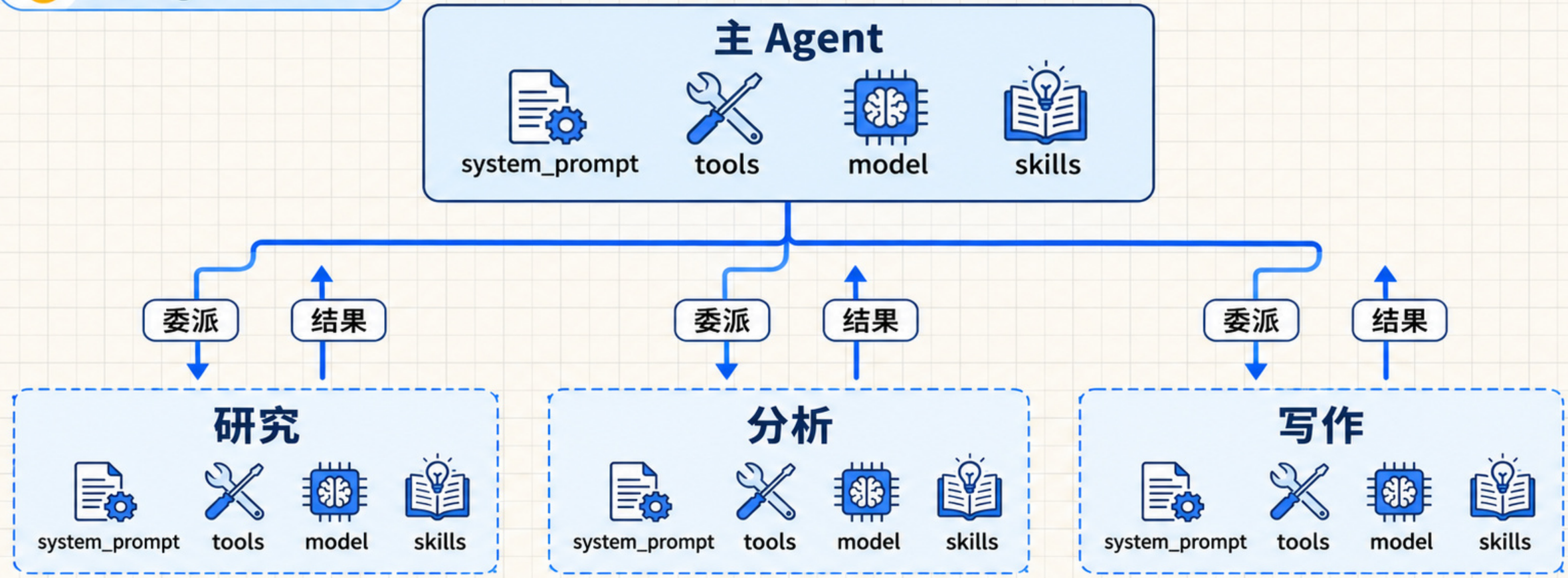
Deep Agents 实战

第5章：子 Agent 与上下文隔离

🎯 让主 Agent 学会委派



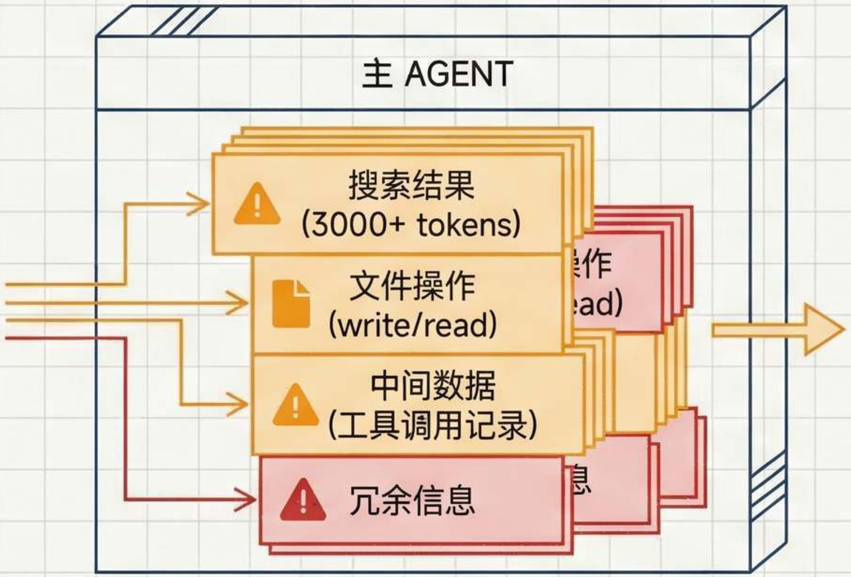
Context Quarantine · v0.7 内容校准

⚠️ 隔离中间对话，不等于隔离共享文件。

上下文膨胀问题

中间过程正在淹没你的主 Agent

膨胀状态（Bloated State）



- 一个研究子任务可能涉及：5 次搜索 + 每次 3000+ tokens 结果
- 多次 write_file 保存中间结果、多次 read_file 回顾整理
- 所有工具调用记录堆积在主 Agent 上下文中

理想状态（Ideal State）



- 主 Agent 其实只需要最终的研究摘要

Context Quarantine：隔离什么？

主 Agent 负责协调，子 Agent 负责专项执行



⚠ 上下文隔离不是安全沙箱；共享文件仍可能相互影响。返回原始大数据也会重新撑大主上下文。

什么时候用子 Agent?

决策速查表

场景	推荐
 需要多次搜索和整理的研究任务	✓ 大量中间结果会膨胀上下文
 需要特殊工具或指令的专业任务	✓ 子 Agent 可有自己的工具集
 需要不同模型能力的任务	✓ 子 Agent 可用不同模型
 需要高层协调的复杂任务	✓ 主 Agent 协调，子 Agent 执行
 单步简单查询	✗ 委派开销大于收益
 需要保留中间上下文的任务	✗ 子 Agent 上下文不回传

字典方式定义子 Agent

独立指令，明确任务边界

```
researcher = {
   "name": "researcher",
   "description": "多源研究并返回摘要与来源",
   "system_prompt": "研究主题，返回500字内摘要。",
   "tools": [internet_search],
   "skills": ["/skills/research/"],
}

agent = create_deep_agent(
   model=model,
   subagents=[researcher],
)
```

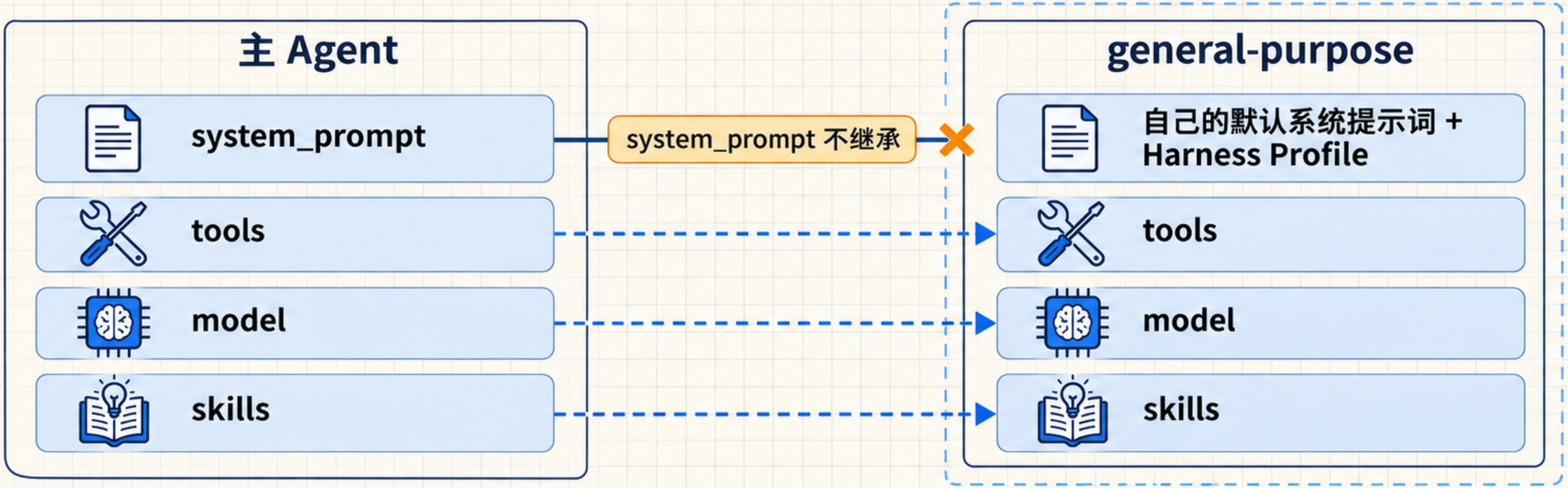


tools 省略时继承业务工具，显式指定则替换；
Harness 文件工具仍由中间件装配。

示意片段：需导入 create_deep_agent；model、internet_search 和技能目录由应用准备。

General-purpose：默认通用子 Agent

复用部分配置，不继承主 Agent 的全部行为



模型可发起的工具调用（示意）：

```
task(
    subagent_type="general-purpose",
    description="研究主题并返回摘要与来源"
)
```

⚠ 可用同名 `spec` 覆盖配置；默认栈同名中间件覆盖可继承，主 Agent 新增的 `Todo` 不自动继承。

配置边界：不要假设自动继承

v0.7 · 默认隔离模式

规划

- 主 Agent：middleware=[TodoListMiddleware()]
- 子 Agent：自己的 middleware 中再启用
- general-purpose 用同名 spec 覆盖
- 双方 todos 不自动共享或写回

权限

- permissions 省略：继承父规则
- 显式提供：整体替换，不是叠加
- 业务 tools 精简不等于撤销内置文件工具
- 核对 Backend、权限与实际工具集

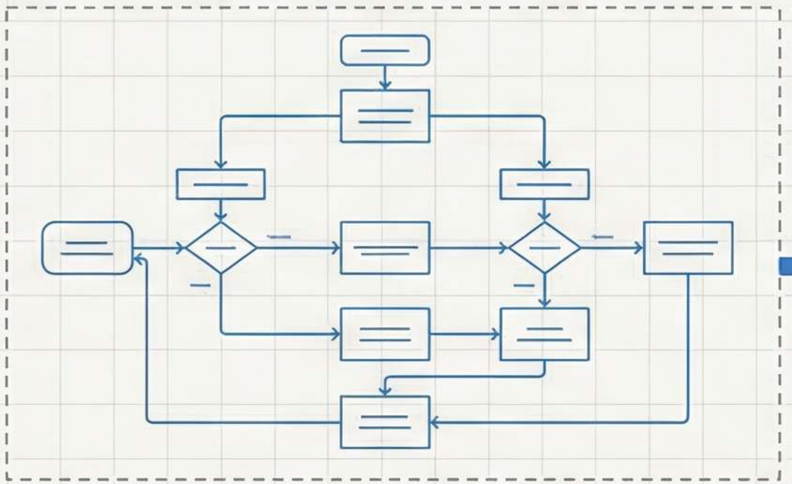
禁用

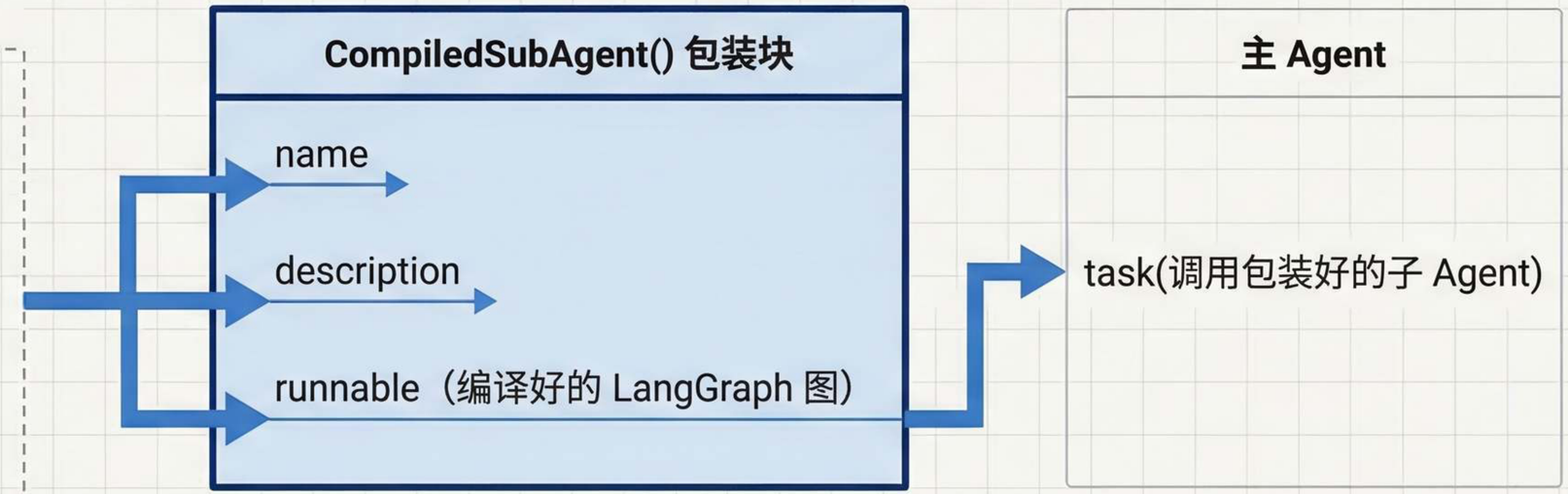
- 对实际模型匹配的 Harness Profile
- general_purpose_subagent.enabled=False
- 且不配置任何同步或异步子 Agent
- 不要用 excluded_middleware 排除 SubAgentMiddleware

以上是默认 isolated 模式；显式 mode="fork" 的提示词与中间件继承规则不同。

CompiledSubAgent
集成 LangGraph 工作流为子 Agent

预构建 LangGraph 工作流（复杂分支逻辑）



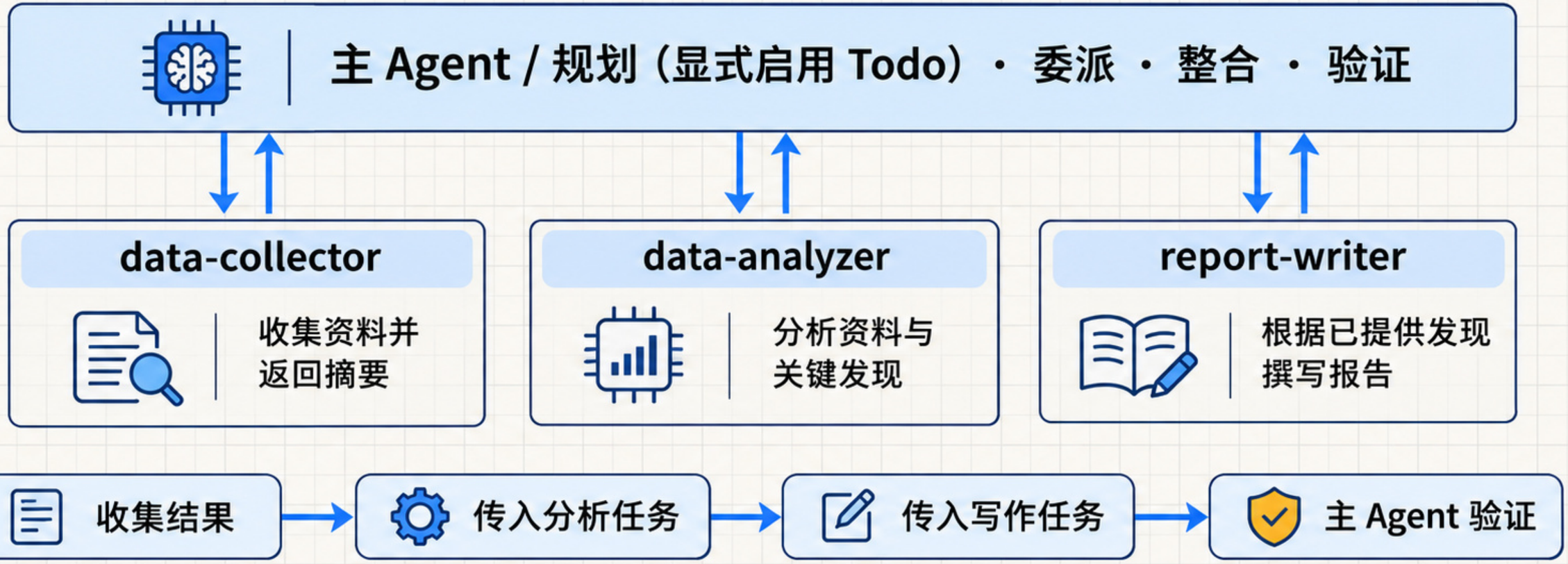


- 适用场景：需要多步骤、有分支逻辑的复杂工作流
- 三个参数：name + description + runnable（编译好的 LangGraph 图）
- 要求：LangGraph 图的 State 必须包含 "messages" 键
- 选择建议：大多数情况用字典方式；有现成 LangGraph 图或需要复杂分支循环时用 CompiledSubAgent

⚠️ 注：State 必须含 "messages" 键

多子 Agent 协作：协调与校验

研究报告的一条示意路径，不是固定执行顺序



⚠ 不要假设子 Agent 自动知道前一个任务结果。把必要数据或可访问文件路径写进 description。

ⓘ 同步 task 与第6章的异步后台任务管理是不同运行方式。

结构化输出：让结果便于处理

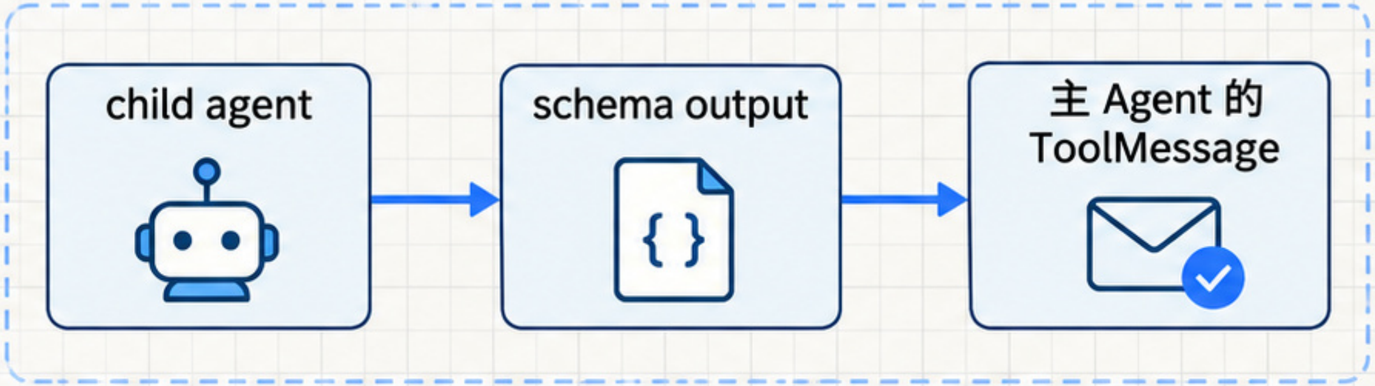
response_format 约定结构，不能替代事实核验

</> 示例 schema:

```python
from pydantic import BaseModel

class Findings(BaseModel):
    summary: str
    sources: list[str]

researcher["response_format"] = Findings
```



默认

最后一条消息的文本

{ } 正常结构化完成

structured_response
序列化为 JSON

⚠ 未生成 structured_response 时可能回退为文本；拒绝与异常要单独处理。JSON 可解析，不代表事实可靠。

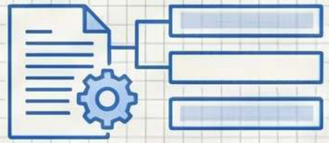
示意片段复用 researcher；应用仍需验证来源与业务约束。

子 Agent 最佳实践

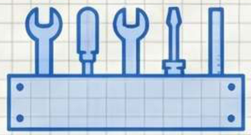
五条关键原则



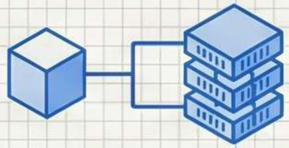
1 **描述要具体** — 主 Agent 靠 description 决定何时委派及委派给谁



2 **System Prompt 要详细** — 包含输出格式要求和字数限制



3 **工具集要精简** — 最小权限原则，只给必要的工具



4 **不同子 Agent 用不同模型** — 轻量任务用小模型，复杂任务用强模型



5 **返回结果要精练** — 只返回核心内容，否则失去上下文隔离的意义

本章回顾与下一步

上下文隔离

- 子任务中间消息留在独立上下文
- 共享文件边界另看 Backend



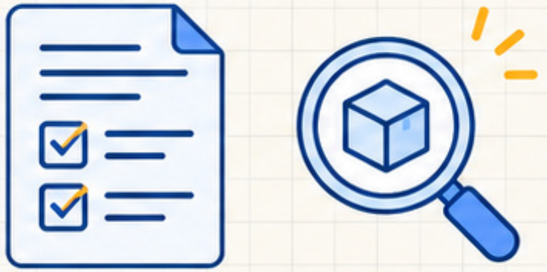
配置清晰

- 专属提示词与明确 description
- Todo、Skills、权限分别核对



结果可验证

- 精炼摘要或结构化结果
- 主 Agent 检查来源与产物



第3章：七个文件工具 → 第4章：显式启用 Todo → 第5章：task 委派

下一章：Async Subagent · 异步委派与后台协同

⚠ 本章可在 Python 进程 invoke；下一章需 Agent Protocol 兼容服务与 thread/run 管理。